

AAKAAR

14-CAPS

Component: Capabilities & Catalog

The library of automation building blocks and how they stay safe

Component

AUDIENCE	Engineers, Solution builders
VERSION	1.0
STATUS	Released
CLASSIFICATION	Confidential

Capabilities & Catalog

In plain terms: a capability is a pre-built, safety-checked Lego brick that the platform can snap into an automation. "Log into a portal," "read a PDF," "send an email," "list a folder over SFTP" — each is one capability. Staff author them once, the platform discovers them automatically, and a tenant only ever sees the ones it has been granted. Because every brick is vetted up front, the people composing automations never have to think about the dangerous edges (a malicious zip file, a request that secretly hits an internal server, a credential leaking into a log) — the brick already handles that.

This document explains what a capability is, how the platform finds them, the safety bar every capability clears, and a full catalog of the bricks that ship today, grouped by domain.

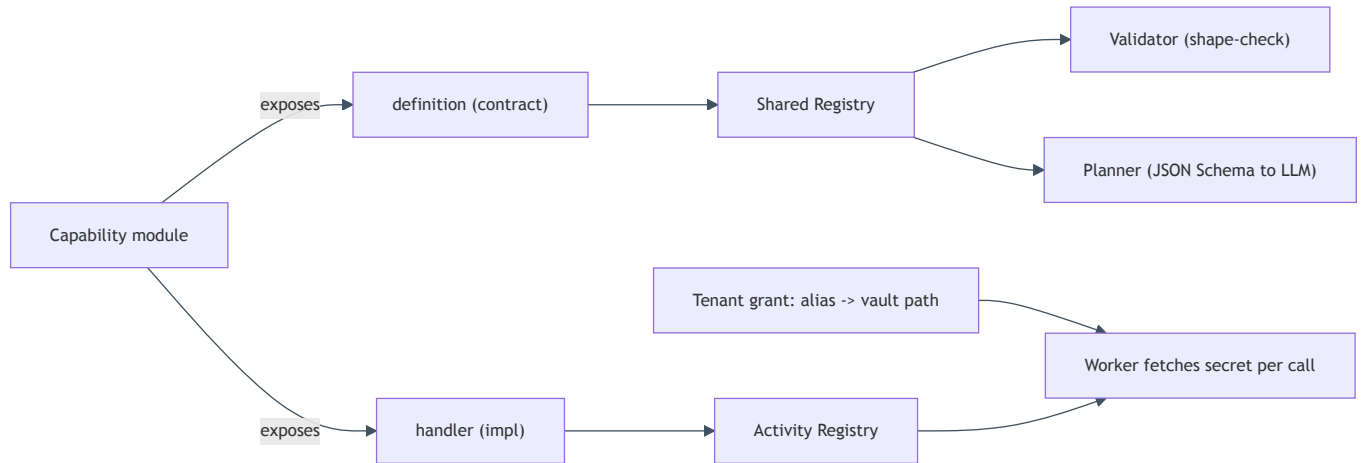
What a capability is

A capability is a Python module that exposes exactly two things: a `definition` (a `CapabilityDefinition` registered in the shared registry) and a `handler` (the async function the engine invokes). The definition is the contract the rest of the system reasons about; the handler is the implementation nobody else needs to see.

The definition declares, for every capability:

Field	Purpose
<code>ref</code>	Stable identifier, always <code>cap.*</code> (e.g. <code>cap.web_login</code>). What a DAG node references.
<code>description</code>	Human/LLM-readable summary used in planning and the catalog.
<code>input_schema</code> / <code>output_schema</code>	Pydantic models — the validator shape-checks inputs and the planner serialises them as JSON Schema for the LLM.
<code>secrets</code>	Names only of credentials the worker will fetch from the vault by alias at run time. Never values.
<code>tags</code>	Soft labels for search and filtering.
<code>side_effecting</code>	Tri-state flag (True / False / None) driving dry-run simulation — see below.

The cardinal rule: **the DAG carries references, never secrets or live data**. A capability declares which credential *names* it needs (`username` , `password` , `api_token`); the tenant's grant binds those names to a vault path; the worker fetches the real values fresh per call and they never flow back into the run environment.

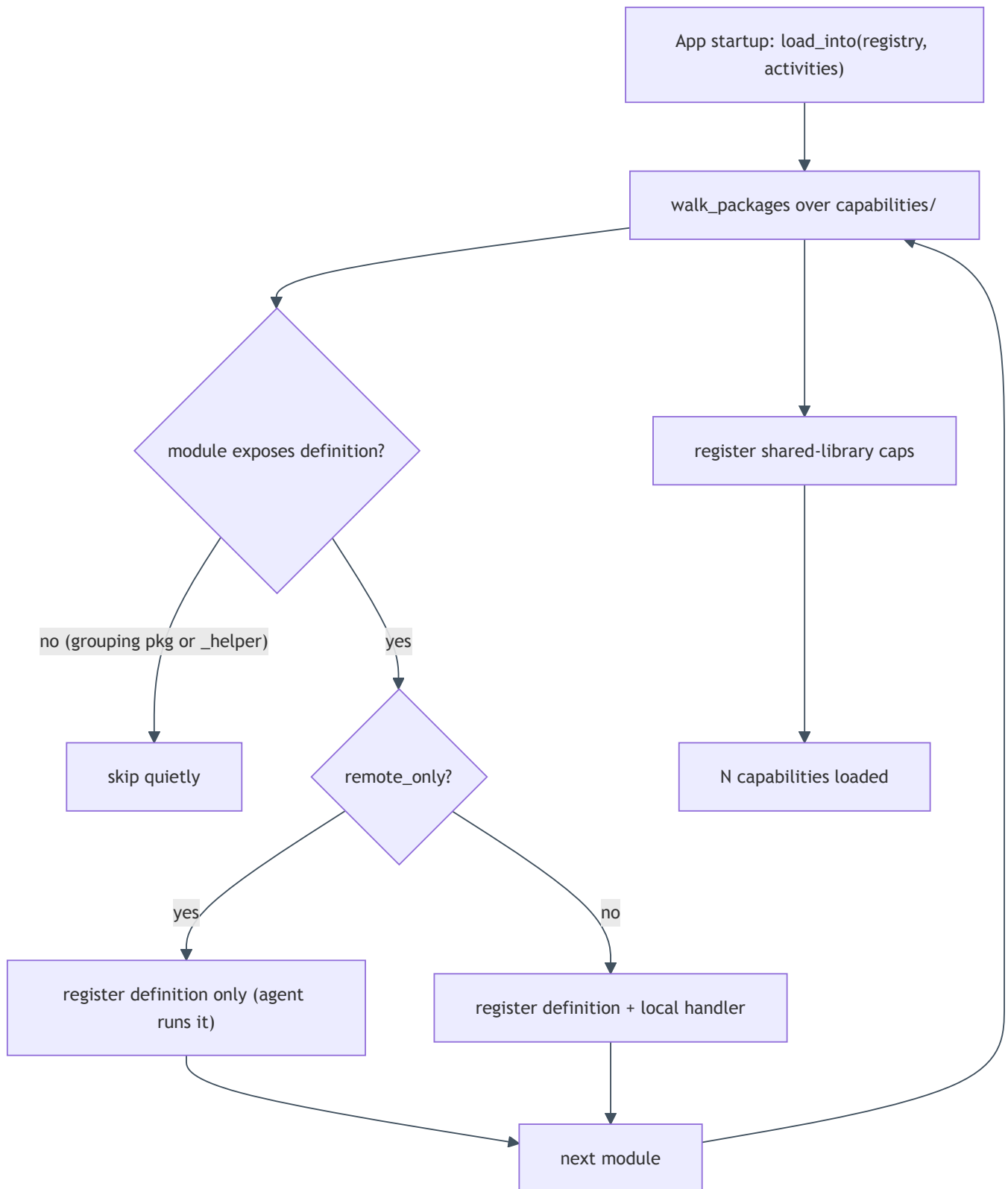


Capabilities sit above generic **actions** (platform primitives like `browser.navigate`) and **control** nodes (`control.branch`, `control.for_each`, `control.wait`, `human.prompt`). The difference that matters: capabilities are *tenant-grantable* and *own credentials*; actions are tenant-agnostic and carry none. If an action would need authentication, that is the signal it should be wrapped as a capability instead.

How capabilities are auto-discovered

There is no central list of capabilities to maintain. At process startup the loader walks the `aakaar.capabilities` package, and any module that exposes a `definition` (and a `handler`, unless it is remote-only) is registered. Drop a folder, restart, the capability is live tenant-side once granted.

Flowchart of the startup discovery walk.



Two rules fall out of how the walk works:

- Modules whose name starts with `_` (helpers like `_shared`, `web_login.discovery`) are skipped — they are infrastructure, not capabilities.
- A capability can be **remote-only**: its definition (schema, tags, placement) is registered for planning and validation, but it has no local handler because the implementation runs on a remote desktop agent. The six `cap.desktop_*` / `cap.window_manage` / `cap.key_send` / `cap.clipboard_write` capabilities work this way.

The safety bar

Every capability that touches the outside world clears the same set of guards. This is the whole point of the capability abstraction: the danger is handled once, in the brick, so it can never be forgotten by whoever composes the workflow.

Guard	What it stops	Where it lives
SSRF guard	Outbound HTTP being tricked into hitting internal services or cloud metadata (<code>169.254.169.254</code>). Resolves every request's host and refuses private / loopback / link-local / reserved / multicast addresses. A grant may pass an <code>allow_hosts</code> allowlist for legitimate LAN calls.	<code>core/net/ssrf.py</code> , used by <code>api_call</code> , <code>webhook_send</code> , extended HTTP
Zip-slip guard	Archive members escaping the extraction root via <code>../</code> , absolute paths, or symlinks (path traversal).	<code>archive_manage</code>
Zip-bomb / decompression-bomb guard	A small archive inflating into a memory bomb. Caps members (<code>_MAX_MEMBERS = 1000</code>) and total uncompressed bytes (<code>_MAX_TOTAL_UNCOMPRESSED = 256 MiB</code>) on list/extract.	<code>archive_manage</code>
Image-bomb guard	A tiny image file decoding into gigapixels. Source pixel count checked before any decode, output geometry checked before allocation, capped at <code>_MAX_PIXELS = 64M</code> , below Pillow's own bomb threshold.	<code>image_convert</code> (and <code>ocr_extract</code>)
Argv-only shell	Shell injection. External tools are invoked with an argument vector, never a string passed to a shell; no <code>shell=True</code> .	shell-invoking capabilities
Credential envelope	Secrets leaking into the DAG, logs, or run env. Capabilities declare credential <i>names</i> ; values are fetched fresh from the vault per call against the tenant grant and never returned to the DAG.	<code>interpreter/credentials.py</code> + <code>_base.py</code>
<code>side_effecting</code> flag	A simulation accidentally moving money. Drives the dry-run path.	every capability definition

The `side_effecting` flag and dry-run

The flag is deliberately tri-state, and the default is the *safe* one:

Value	Meaning	Dry-run behaviour
<code>True</code>	Performs an external, irreversible effect (send / write / upload / transfer).	Always simulated to a fake success.
<code>False</code>	Read-only (scrape, read, validate, GET-style).	Runs for real , even in a dry-run.
<code>None</code>	Undeclared.	Treated conservatively as side-effecting — simulated — so a new capability that forgot to declare can never move money during a rehearsal.

`None` is the safe fallback, not a recommendation. Authors should declare the flag explicitly. The point is that forgetting fails *closed*: an undeclared capability is simulated, never executed, in a dry-run.

The catalog

The following capabilities ship today, grouped by domain. Refs and tags are taken directly from the registered definitions. The "Side effect" column reflects the `side_effecting` declaration; capabilities reaching arbitrary URLs go through the SSRF guard.

Web & browser

Drive a real browser session: log in, navigate, scrape, fill forms, screenshot.

Ref	What it does	Side effect
<code>cap.open_url</code>	Open a fresh browser session and navigate to a URL.	No
<code>cap.web_login</code>	Log into an arbitrary web application (owns credentials).	No (auth)
<code>cap.web_navigate</code>	Drive a browser session through a sequence of steps.	Varies
<code>cap.web_scrape</code>	Read a web page and return its content, optionally LLM-structured.	No
<code>cap.web_form_fill</code>	Fill (and optionally submit) a web form.	Yes (on submit)
<code>cap.screenshot</code>	Capture a screenshot of the current page or a single element.	No
<code>cap.file_download</code>	Download a file through an authenticated browser session.	No
<code>cap.file_upload</code>	Attach a managed-storage file to a file input.	Yes

Files & SFTP

Object-store file operations and remote file movement over SFTP-over-SSH.

Ref	What it does	Side effect
<code>cap.file_manage</code>	Object-store file operations (copy / move / delete in managed storage).	Yes
<code>cap.file_watch</code>	Bounded poll of object storage for create / modify / delete.	No
<code>cap.archive_manage</code>	Create / extract / list zip and tar archives (zip-slip + bomb guarded).	Yes (extract/create)
<code>cap.sftp_login</code>	Establish an authenticated SFTP-over-SSH session (owns credentials).	No (auth)
<code>cap.sftp_list</code>	List a remote directory over an SFTP session.	No
<code>cap.sftp_read</code>	Pull a file off an SFTP server into managed storage.	No
<code>cap.sftp_write</code>	Push a file from managed storage onto an SFTP server.	Yes
<code>cap.sftp_transfer</code>	Move / copy a file between two paths on SFTP.	Yes

Data & documents

Tabular data, spreadsheets, SQL, PDFs, images and OCR.

Ref	What it does	Side effect
<code>cap.data_transform</code>	Apply a pipeline of tabular transforms with pandas.	No
<code>cap.data_validate</code>	Validate tabular records against a simple field schema (recon checks).	No
<code>cap.db_query</code>	Run a parameterized SQL query against a relational database.	Varies
<code>cap.spreadsheet_read</code>	Read an xlsx / csv spreadsheet into structured rows.	No
<code>cap.doc_extract</code>	Read a stored document and return its structured content.	No
<code>cap.pdf_extract</code>	Pull text out of a stored PDF, whole or per-page.	No
<code>cap.pdf_tools</code>	Page-level PDF operations with pypdf (split / merge / select).	Yes
<code>cap.image_convert</code>	Image transforms over a stored object via Pillow (bomb-guarded).	Yes
<code>cap.ocr_extract</code>	OCR an image stored in object storage into text.	No

Comms (email)

IMAP fetch, SMTP send, and structured parsing of messages.

Ref	What it does	Side effect
<code>cap.email_fetch</code>	Fetch recent messages from an IMAP mailbox (owns credentials).	No
<code>cap.email_send</code>	Send an email over SMTP (stdlib smtpplib).	Yes
<code>cap.email_parse</code>	Turn an email/text body into structured data via the runtime LLM.	No
<code>cap.eml_parse</code>	Structural parse of a raw RFC822/EML message, including attachments.	No

Integration (HTTP)

Outbound API calls and webhooks, both behind the SSRF guard.

Ref	What it does	Side effect
<code>cap.api_call</code>	Authenticated outbound HTTP request (SSRF-guarded).	Varies
<code>cap.webhook_send</code>	POST a JSON payload to an outbound webhook URL (SSRF-guarded).	Yes

Remote / desktop (agent-only)

These are **remote-only** contracts: the definition is registered for planning, but the implementation runs on an enrolled remote desktop agent. They drive RPA on legacy or thick-client apps that have no API.

Ref	What it does	Side effect
<code>cap.desktop_click</code>	Click on the remote desktop by coordinates or matched image.	Yes
<code>cap.desktop_type</code>	Type text into the focused window on the remote desktop.	Yes
<code>cap.desktop_scroll</code>	Scroll the focused window on the remote desktop.	Yes
<code>cap.key_send</code>	Press a validated key combo on the remote desktop.	Yes
<code>cap.clipboard_write</code>	Write text to the remote machine clipboard.	Yes
<code>cap.window_manage</code>	List or manipulate windows on the remote desktop.	Yes

A banking example

Consider a **reconciliation** workflow pulling a settlement file from a counterparty and checking it. It might compose: `cap.sftp_login` → `cap.sftp_read` (fetch the file into managed storage) → `cap.spreadsheet_read` (parse it) → `cap.data_validate` (assert the schema and totals) → `cap.email_send` (notify ops of breaks). In a dry-run, the login, read, parse and validate steps run for real (they are read-only) while `cap.email_send` is simulated — so an operator can rehearse the entire recon, see the real validation result, and send nothing.

Reading the catalog this way is the fastest route to authoring: pick the read-only bricks for the "look at the data" half, pick the side-effecting bricks for the "act on it" half, and remember that the second half is exactly what a dry-run will hold back.

Where this fits

Capabilities are the vocabulary the Workflow Authoring Guide composes into a DAG; the credentials they reference live in the vault described in the Security doc; the engine that invokes their handlers layer-by-layer is covered in the Backend & API component. To see which capabilities a tenant actually has, an operator opens the **Capabilities** page in the Web Console.